Python OOP Interview Questions (Most Commonly Asked)

1. What is OOP in Python? Explain its main pillars.
2. What is a class and an object in Python? Explain with an example.
3. Explain the __init__ method and the role of self parameter.
4. What is Encapsulation? How is it implemented in Python?
5. Explain Public, Protected, and Private access modifiers in Python with examples.
6. What are getters and setters in Python? Why are they used?
7. What is Abstraction? How do you implement it using abstract classes in Python?
8. What is the difference between abstract class and normal class?
9. Explain Inheritance in Python. What are the different types of Inheritance?
10. What is Multiple Inheritance? Explain with an example and mention any issues (MRO).
11. What is Polymorphism? Explain Method Overriding and Method Overloading in Python.
12. What is Duck Typing in Python?
13. Explain Operator Overloading with an example using Dunder methods.
14. What are Dunder (Magic) methods? Give examples of commonly used ones.
15. What is the difference between __str__ and __repr__?
16. Explain Class methods and Static methods. When to use @classmethod and @staticmethod?
17. What is the difference between instance method, class method, and static method?
18. What are decorators in Python? Explain with @classmethod example.
19. Explain Shallow Copy vs Deep Copy with examples.
20. What is Method Resolution Order (MRO) in Python?
21. How does Python support multiple inheritance? Explain super() function.
22. What is the difference between is and == operator?
23. Explain lambda functions with examples.
24. What are List Comprehensions? Give examples.
25. Explain *args and **kwargs with examples.

26. What is the difference between print() and return in functions?
27. What are Python keywords? Can we use them as variable names?
28. Explain Global and Local variables in Python.
29. What is Dynamic Typing in Python?
30. Explain the difference between List, Tuple, Set, and Dictionary.
31. What are built-in functions you frequently use (len, type, id, etc.)?
32. What is Garbage Collection in Python?
33. Explain the difference between copy.copy() and copy.deepcopy().
34. What is the purpose of __del__ method?
35. How do you achieve data hiding in Python?
36. What is the benefit of using properties (@property) instead of traditional getters/setters?
37. Explain the difference between Composition and Inheritance.
38. What is an abstract method? How do you define it?
39. Can we create an object of an abstract class in Python?
40. What happens if a child class does not implement an abstract method from parent?